

FORMATION EMBARQUÉE

Guide du formateur

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Guide du formateur

À qui s'adresse ce document : à toute personne qui **anime** cette formation (enseignant, formateur en centre, tuteur d'alternance, ou apprenant en autodidacte qui veut se structurer). Il donne le planning, le matériel à prévoir, les modalités d'évaluation et les conseils d'animation que les modules eux-mêmes ne contiennent pas.

1. Philosophie de la formation

Trois principes guident tout le contenu — respecte-les en animant :

1. **La pratique d'abord.** Ratio cible : 1 h de théorie pour 2-3 h de mains dans le cambouis. Un apprenant qui a seulement /u le chapitre pointeurs ne sait pas les utiliser.
2. **Le C comme socle.** Tout en dépend (Arduino = C++, SCL $\approx$ Pascal/C, registres STM32 = pointeurs). Ne laisse personne « sauter » le module 01.
3. **Le fil rouge conceptuel.** Le même problème (ring buffer, machine d'états, hystérésis, feu tricolore, UART) revient en C, C++, VHDL, SCL. **Fais expliciter ces correspondances** aux apprenants : c'est là que se construit la vraie compétence transférable.

Signal de réussite : un apprenant qui, devant un problème neuf, dit « c'est la même FSM qu'au TD 01, mais en VHDL » a compris l'essentiel.

2. Formats possibles

La formation (~190 h) s'adapte à plusieurs cadres :

Format	Durée	Public	Découpage conseillé
Autodidacte	8-12 mois, ~6 h/sem.	reconversion, curieux	suivre le plan du module 09
Module intensif	8 semaines, 30 h/sem.	bootcamp, reconversion pro	modules 00-03 + 10 + TP 1/5
Semestre universitaire	14 semaines $\times$ 4 h	BTS/BUT/licence	1 module + son TD par semaine
Formation continue	sessions de 3-5 jours	salariés en poste	1 thème par session (voir §7)
Alternance / tutorat	fil de l'année	apprenti + tuteur	TP alignés sur les projets de l'entreprise

Le découpage en cours / TD / TP / évaluations est fait pour qu'on puisse **piocher** : un module d'automatisme seul (00, 07, 08) est cohérent pour un public électrotechnique ; un parcours firmware (00-03, 06, 10) l'est pour un public informatique.

3. Planning type — semestre de 14 semaines

Exemple pour un cadre scolaire (4 h/semaine encadrées + travail perso) :

Sem.	Cours (2 h)	Séance encadrée (2 h)	À finir en autonomie
1	Module 00 §1-4	TD 00 (conversions, bus)	QCM module 00
2	Module 00 §5-8 + Module 01 §1-3	TD 01 ex. 1-2 (bits, ring buffer)	lire 01 §4-7
3	Module 01 §6-9 (pointeurs !)	TD 01 ex. 3-4 (FSM, trame)	compiler <code>code/c/</code> , QCM 01
4	Module 02 (C++/RAII)	TD 02 ex. 1-3	TP 1 séance 1 (à distance/labo)
5	Module 03 (Arduino)	TP 1 séance 2 (OLED, modules)	TP 1 séance 3
6	Module 03 (suite) + revue TP 1	TP 1 séance 4 (ESP32/MQTT)	finir TP 1, remplir grille
7	Module 04 (VHDL) §1-4	TD 04 ex. 1-2 (mux, BCD)	installer GHDL, QCM 04
8	Module 04 §5-8	TP 2 séances 1-2 (UART TX/RX)	TP 2 séance 3
9	Module 05 (Java)	TD 05 (décodeur, prod/cons)	compiler <code>code/java/</code>
10	Module 07 (Siemens) §1-6	TD 07 ex. 1-2 (auto-maintien, FB)	installer TIA essai
11	Module 07 §7-11	TP 3 séances 1-2 (analyse, prog.)	TP 3 séance 3 (PLCSIM)
12	Module 08 (Schneider)	TP 3 séance 4 (HMI) + TD 08	TP 4 (au fil)
13	Module 10 (STM32)	TP 5 séances 1-2 (socle, driver)	TP 5 séances 3-4
14	Module 06 + révisions	Lancement projet d'évaluation	projet (hors planning)

Adapte selon ton public : électrotechnique → étale 07-08 sur plus de semaines et allège 04 ; informatique → l'inverse.

4. Matériel à prévoir

4.1 Par poste / binôme (travaux pratiques matériels)

Élément	Qté/poste	Prix unit.	Sert aux
Arduino Uno/Nano (ou clone)	1	5-25 €	TP 1 (séances 1-3)
ESP32 DevKit	1	8-12 €	TP 1 (séance 4)
Breadboard + kit fils	1	8 €	tous les TP matériels
DHT22, OLED SSD1306, HC-SR04, potentiomètres, LED, boutons, résistances	1 kit	20-25 €	TP 1
Nucleo-F411RE	1	15-20 €	TP 5
BME280 (I2C)	1	5 €	TP 5 (driver maison)
Analyseur logique 8 voies (clone)	1 pour 2-3 postes	10 €	voir I2C/SPI/UART en vrai — très pédagogique
Multimètre	1 pour 2 postes	20 €	dépannage

Budget matériel : ~70-90 € par poste. Un analyseur logique partagé (avec PulseView/sigrok) transforme la compréhension des protocoles du module 00 — investis-y même à un pour trois postes.

4.2 FPGA (TP 2) — optionnel

- **Par défaut : rien à acheter.** GHDL + GTKWave (gratuits) couvrent tout le TP 2 en simulation. C'est la voie recommandée pour débiter.
- Pour aller sur carte : Tang Nano 9K (~15 €) ou Basys 3 (~150 €, budget établissement). Une carte pour 3-4 postes suffit (la synthèse se fait à tour de rôle).

4.3 Automates (TP 3-4) — aucun achat nécessaire

- **Siemens** : PLCSIM (inclus dans TIA Portal) simule la CPU. Licence d'essai 21 jours, ou licences éducation SCE.
- **Schneider** : Machine Expert Basic est **gratuit** avec simulateur intégré ; Control Expert en version d'essai.
- Un vrai automate (M221 ou S7-1200 d'occasion, ~100-200 €) est un *plus* pédagogique mais non requis — garde-le pour une démo commune.

4.4 Logiciels à installer (tous gratuits ou en essai)

Logiciel	Pour	Note
GCC / G++ / Make	modules 01-02, 06	<code>build-essential</code> sous Linux, MinGW/WSL sous Windows
IDE Arduino 2.x ou PlatformIO	module 03, TP 1	+ Wokwi (navigateur) en secours
GHDL + GTKWave	module 04, TP 2	<code>apt install ghdl gtkwave</code>
JDK 17+	module 05	Temurin/OpenJDK
STM32CubeIDE	module 10, TP 5	gratuit, ~3 Go
TIA Portal + PLCSIM	module 07, TP 3	essai 21 j ; PC costaud requis
Machine Expert Basic	module 08, TP 4	gratuit
Python 3 + pymodbus	TP 4, scripts	<code>pip install pymodbus</code>

Prépare une VM ou une image disque avec tout d'installé si tu as plusieurs postes — l'installation de TIA Portal seule prend une demi-journée.

5. Modalités d'évaluation

5.1 Trois niveaux de contrôle

1. **QCM par module** (`evaluations/qcm.md` , 56 questions) : contrôle de connaissances, en autonomie.
Seuil de passage **80 %** avant le module suivant. Sert de filtre : un apprenant sous 80 % n'est pas prêt.
2. **Grilles de TP /20** (dans chaque TP) : évaluent la mise en œuvre. À remplir **avec preuves** (captures, code, historique Git).
3. **Projet d'évaluation finale** (`evaluations/projets-notes.md` , 3 sujets /100) : le contrôle terminal, 30-40 h. Seuil de réussite **70**.

5.2 Barème global suggéré (cadre certifiant)

Composante	Poids
QCM (moyenne des modules suivis)	20 %
TP (moyenne des grilles /20)	40 %
Projet final /100	40 %

5.3 Ce qu'un formateur doit valoriser (et pénaliser)

Valorise (au-delà du « ça marche ») : - la **gestion des cas de panne** (capteur débranché, réseau coupé, entrée hostile) — c'est le marqueur n°1 du niveau ; - l'**architecture** (modules séparés, sorties centralisées, ISR courtes) ; - la **démarche** (spec/GRAFCET avant le code, recette de test écrite) ; - l'**honnêteté du journal de bord** (impasses comprises).

Pénalise systématiquement : - une sécurité qui vit dans l'IHM ou dans une séquence (au lieu d'en aval) ; - un `delay()` bloquant dans une boucle qui doit rester réactive ; - du code qui « marche » mais qu'on ne peut pas expliquer ; - l'absence de test des cas limites.

Un TP qui marche parfaitement en conditions nominales mais gèle au premier imprévu vaut **moins** qu'un TP incomplet dont chaque partie est robuste et comprise.

6. Conseils d'animation

6.1 Les moments où les apprenants décrochent (anticipe-les)

Point de blocage	Où	Remède d'animation
Les pointeurs	module 01 §6	dessine la mémoire au tableau ; fais tracer adresse/valeur à la main avant tout code
« Le VHDL n'est pas séquentiel »	module 04	insiste : « quel circuit je décris ? » ; interdis le mot « exécuter »
Latch involontaire	module 04, TD 04	montre le rapport de synthèse ; règle : « affecte la sortie dans TOUTES les branches »
L'octet signé en Java	module 05, TD 05	fais afficher <code>(byte)0xA5</code> : « pourquoi -91 ? »
Sécurité dans l'IHM	modules 07-08	scénario : « la liaison HMI tombe, le bouton reste à 1 — que se passe-t-il ? »
RCC oublié (périphérique muet)	module 10	réflexe à marteler : « horloge du périphérique AVANT tout accès »

6.2 Techniques qui marchent

- **Casser le code corrigé.** Fais enlever un `volatile`, inverser une condition, et observer quel test échoue. On apprend plus en cassant qu'en lisant (c'est explicite dans `code/README.md`).
- **Le débogueur comme outil pédagogique.** Sur STM32, montre le tableau ADC qui bouge CPU arrêté (DMA), ou le Fault Analyzer sur un pointeur nul. Ces « waouh » ancrent les concepts.
- **Faire dessiner avant de coder.** Aucune FSM, aucun GRAFCET ne se code avant d'être dessiné. Exige le schéma en photo dans le dépôt.
- **Revue de code entre pairs.** Fais relire le TP d'un binôme par un autre avec la grille — apprendre à lire du code est aussi important qu'à en écrire.

6.3 Gestion de l'hétérogénéité

Les groupes mélangent souvent profils électrotechnique et informatique. - Les « info » filent en C/C++/VHDL mais butent sur le câblage et les sécurités machine → binôme-les avec des « électro ». - Les « électro » sont à l'aise en LADDER/GRAFCET mais rament sur les pointeurs → binômes croisés sur les TP. - Prévois des **extensions** (« pour aller plus loin » de chaque TP) pour les plus rapides pendant que les autres consolident.

7. Découpage en sessions courtes (formation continue)

Pour des salariés, en modules de 3-5 jours autonomes :

Session	Jours	Contenu	Public visé
« C embarqué »	3	modules 00-01 + TD	dev voulant descendre bas niveau
« Arduino/IoT »	3	modules 03, 05 §7 + TP 1	prototypage, makers pro
« FPGA/VHDL initiation »	4	module 04 + TP 2	électroniciens
« Automatismes Siemens »	5	modules 00, 07 + TP 3	techniciens de maintenance
« Automatismes Schneider + supervision »	5	module 08 + TP 4	automaticiens
« STM32 firmware pro »	5	modules 06, 10 + TP 5	dev embarqué

Chaque session est autoportante grâce aux prérequis indiqués en tête de module — un rappel d'une demi-journée sur le module 00 suffit à ouvrir n'importe quelle session à un public un peu éloigné.

8. Checklist de préparation (avant la première séance)

- [] Postes équipés / VM prête avec les logiciels de la §4.4.
- [] Kits matériel montés et **testés** (un DHT22 mort en début de TP fait perdre une heure au groupe).
- [] Comptes créés : GitHub (chaque apprenant versionne dès le blink), Wokwi si pas de matériel.
- [] Le dépôt de la formation cloné et accessible aux apprenants.
- [] Licences TIA Portal activées (le délai d'activation peut surprendre).
- [] Toi-même : avoir fait **au moins le TP** de chaque module que tu encadres. On n'anime bien qu'un TP qu'on a soi-même buté et débogué.

9. Pour finir

Cette formation n'est pas un livre à réciter : c'est un **atelier**. Ton rôle de formateur est moins de transmettre des faits (ils sont dans les modules) que de : - provoquer les « waouh » (débugueur, analyseur logique, boucle 256/256) ; - faire expliciter les correspondances entre langages ; - exiger la robustesse et la démarche, pas seulement le résultat ; - accompagner chacun jusqu'à son **portfolio de projets finis** — le seul livrable qui compte vraiment pour la suite (emploi, alternance).

Bonne formation. Et souviens-toi de la phrase qui résume tout le cursus : **construire des petits projets qui marchent, souvent.**

➡ Retour à l'[index de la formation](#).